Introdução à criptografia

SEG786203 – CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

INSTITUTO FEDERAL
Santa Catarina
Campus São José

Licenciamento

Slides licenciados sob Creative Commons “Atribuição 4.0 Internacional”

Sumário

1 Introdução

2 Criptografia clássica

3 Criptografia moderna

4 Comunicação segura entre partes

5 Função de dispersão criptográfica

Introdução

Objetivos dessa aula

- Entender criptografia moderna requer aprofundamento em conceitos matemáticos
- Para esse curso a criptografia será apresentada como uma ferramenta de suporte à segurança da informação
- Visão geral dos principais conceitos e técnicas de criptografia que são relevantes para a segurança da informação

Propriedades básicas de segurança

Confidentiality, Integrity, Availability (CIA triad)

Fonte: Operational Technology Cybersecurity for Energy Systems

- **Confidencialidade**
  - A informação só deve ser revelada para usuários autorizados
- **Integridade**
  - A informação não deve ser alterada, destruída ou perdida, de maneira não autorizada ou acidental
- **Disponibilidade**
  - A informação deve estar disponível quando necessário

É necessário garantir as três propriedades para que a informação seja considerada segura

Garanta as propriedades básicas de segurança I

Problemas

1. Você precisa **guardar suas senhas pessoais em um arquivo no seu computador**
2. Você precisa **enviar um arquivo por email para um amigo**
3. Você precisa **armazenar as senhas de seus usuários em um banco de dados**

Exercício em duplas (20 minutos)

- Proponha uma solução para cada problema
- Quais propriedades de segurança estão envolvidas e como elas podem ser garantidas?
- No final, preparem para compartilhar suas respostas com a turma

Problema 1

Discussão

Você precisa **guardar suas senhas pessoais em um arquivo no seu computador**

Problema 1

Discussão

Você precisa **guardar suas senhas pessoais em um arquivo no seu computador**

- **Salvar o arquivo com um nome que não chame a atenção?**
  - `senhas.txt` $\rightarrow$ `receita-bolo-quiabo.txt`

Problema 1

Discussão

Você precisa **guardar suas senhas pessoais em um arquivo no seu computador**

- **Salvar o arquivo com um nome que não chame a atenção?**
  - `senhas.txt` $\rightarrow$ `receita-bolo-quiabo.txt`
- **Criar um ZIP protegido por senha?**
  - `zip -e senhas.zip senhas.txt`

Problema 1

Discussão

Você precisa **guardar suas senhas pessoais em um arquivo no seu computador**

- **Salvar o arquivo com um nome que não chame a atenção?**
  - `senhas.txt` → `receita-bolo-quiabo.txt`
- **Criar um ZIP protegido por senha?**
  - `zip -e senhas.zip senhas.txt`
- **Usar um gerenciador de senhas?**
  - Bitwarden, KeePass, ProtonPass, Google, Apple, etc.

Problema 2

Discussão

Você precisa **enviar um arquivo por email para um amigo**

Problema 2

Discussão

Você precisa **enviar um arquivo por email para um amigo**

- **Criar um ZIP protegido por senha e enviar o ZIP por email?**
  - Como irá compartilhar a senha com o amigo?

Problema 2

Discussão

Você precisa **enviar um arquivo por email para um amigo**

- **Criar um ZIP protegido por senha e enviar o ZIP por email?**
  - Como irá compartilhar a senha com o amigo?
- **Enviar por email o arquivo e o resumo criptográfico sobre o mesmo?**
  - O que garante que o resumo criptográfico não foi modificado?

Problema 2

Discussão

Você precisa **enviar um arquivo por email para um amigo**

- **Criar um ZIP protegido por senha e enviar o ZIP por email?**
  - Como irá compartilhar a senha com o amigo?
- **Enviar por email o arquivo e o resumo criptográfico sobre o mesmo?**
  - O que garante que o resumo criptográfico não foi modificado?
- **Compartilhar previamente um segredo com o amigo e usar o segredo para gerar um resumo criptográfico sobre o arquivo?**
  - Como compartilhar o segredo com o amigo?

Problema 3

Discussão

Você precisa **armazenar as senhas de seus usuários em um banco de dados**

Problema 3

Discussão

Você precisa **armazenar as senhas de seus usuários em um banco de dados**

- **Salvar as senhas em texto claro no banco de dados?**
  - Se o banco de dados for comprometido, as senhas estarão expostas

Problema 3

Discussão

Você precisa **armazenar as senhas de seus usuários em um banco de dados**

- **Salvar as senhas em texto claro no banco de dados?**
  - Se o banco de dados for comprometido, as senhas estarão expostas
- **Salvar as senhas criptografadas no banco de dados?**
  - Se o banco de dados for comprometido, as senhas estarão seguras
  - Mas é necessário garantir a confidencialidade da chave criptográfica

Problema 3

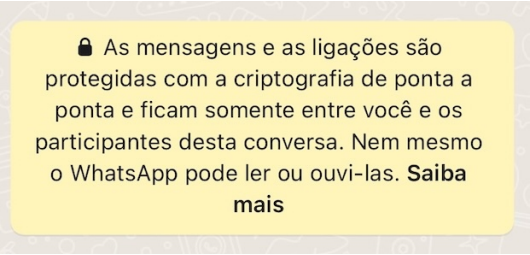
Discussão

Você precisa **armazenar as senhas de seus usuários em um banco de dados**

- **Salvar as senhas em texto claro no banco de dados?**
  - Se o banco de dados for comprometido, as senhas estarão expostas
- **Salvar as senhas criptografadas no banco de dados?**
  - Se o banco de dados for comprometido, as senhas estarão seguras
  - Mas é necessário garantir a confidencialidade da chave criptográfica
- **Salvar resumos criptográficos das senhas no banco de dados?**
  - Se o banco de dados for comprometido, as senhas estarão seguras
  - Mas é necessário uso de funções de dispersão criptográfica seguras e *salt* aleatório para cada senha

Criptografia

- A palavra “criptografia” é oriunda do grego e significa “escrita secreta”
- Consiste na prática e no estudo de técnicas para comunicação segura na presença de adversários
- Pode ser usada para garantir a confidencialidade, a autenticidade e a integridade da informação



Fonte: Captura de tela do aplicativo WhatsApp

Criptosistema

Componente básico da criptografia, consiste em uma tupla de cinco elementos: $(\mathcal{M}, \mathcal{K}, \mathcal{C}, \mathcal{E}, \mathcal{D})$

- $\mathcal{M}$ é o conjunto de mensagens em texto claro
- $\mathcal{K}$ é o conjunto de chaves
- $\mathcal{C}$ é o conjunto de mensagens cifradas
- $\mathcal{E}$ é o algoritmo para cifrar, $\mathcal{E} : \mathcal{M} \times \mathcal{K} \to \mathcal{C}$
- $\mathcal{D}$ é o algoritmo para decifrar, $\mathcal{D} : \mathcal{C} \times \mathcal{K} \to \mathcal{M}$

Criptoanálise

Técnicas para quebrar criptosistemas com o objetivo de recuperar a mensagem original sem conhecer a chave ou o algoritmo de cifragem

- **Ataque com o texto cifrado**
  - adversário possui o texto cifrado e através dele tenta obter o texto em claro ou até mesmo a chave
- **Ataque com o texto em claro**
  - adversário possui o texto cifrado e o texto em claro tendo o objetivo de descobrir a chave

Criptografia clássica

Criptografia clássica

- Na antiguidade a criptografia tinha como **objetivo principal** garantir a **confidencialidade** das mensagens
- A chave era compartilhada entre o emissor e o receptor
- A segurança do sistema dependia do sigilo da chave e do sigilo do algoritmo
- **Exemplos**
  - Cifra de substituição
  - Cifra de transposição

Criptografia clássica

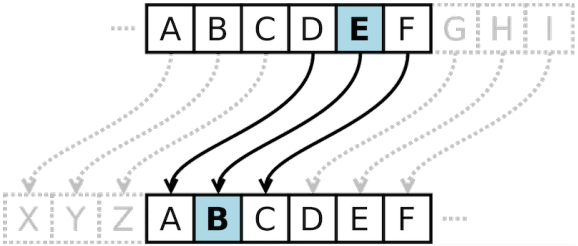
Cifra de substituição

- Os símbolos do texto claro são substituídos por outros símbolos
- Podem ser monoalfabéticas ou polialfabéticas
- Exemplos
  - Cifra de César
  - Cifra de Vigenère
  - Cifra de Enigma

Cifra de César

Exemplo de cifra de substituição monoalfabética

- Uma das técnicas mais simples para cifrar mensagens que foi usada por Júlio César na Roma antiga
- Usa um alfabeto deslocado de n posições para cifrar o texto claro



Fonte: Wikipedia

```
Chave:          3 para esquerda
Texto claro:    IFSC
Texto cifrado:  FCPZ
```

$$\mathcal{E}_n(x) = (x - n) \mod 26 \quad (1)$$

$$\mathcal{D}_n(x) = (x + n) \mod 26 \quad (2)$$

, sendo x a posição da letra no alfabeto e n o deslocamento

Quebra de cifras de substituição monoalfabética

- A cifra de César é facilmente quebrada por um ataque de força bruta
  - Existem apenas 25 chaves possíveis (deslocamentos de 1 a 25)
- Análise de frequência de letras é uma técnica comum para quebrar cifras de substituição monoalfabética
  - Em português a letra mais frequentemente usada é a vogal "a" (14,63%)[1]

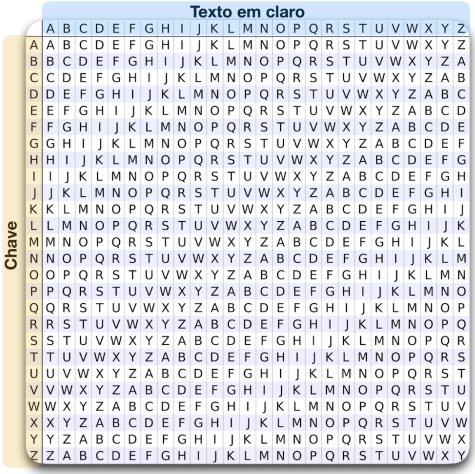
[1] https://pt.wikipedia.org/wiki/Alfabeto_portugu%C3%AAs

Cifra de Vigenère

Exemplo de cifra de substituição polialfabética

- Usa uma palavra-chave para cifrar o texto em claro
- A palavra-chave é repetida até que tenha o mesmo tamanho do texto em claro
- A cifragem é feita letra a letra usando a cifra de César e a letra correspondente da palavra-chave

```
Chave: IFSC
Palava-chave:      IFSCIFSCI FSC
Texto em claro:    SEGURANCA ADS
Texto cifrado:     AJYWZFFEI FVU
```



Fonte: Wikipedia

Enigma

- Máquina eletromecânica com rotores que faz uso de cifra polialfabética
- Patenteada por Arthur Scherbius em 1918 e utilizada pela Alemanha nazista durante a Segunda Guerra Mundial (versão militar em 1928)
- Em 1933 o código foi quebrado por matemáticos poloneses
- Em 1940, Alan Turing e sua equipe conseguiram acelerar a quebra do código

Fonte: Acervo pessoal

Criptografia clássica

Cifras de transposição

- Os símbolos do texto claro são rearranjados (permutados), ou seja, a ordem dos símbolos é alterada
- Exemplos
  - Cítala
  - *Rail fence*

Cítala usada pelos espartanos

Fonte: https://en.wikipedia.org/wiki/Scytale

Uma cerca com 3 trilhos (*rail fence*)

Fonte: https://rocketfence.com/

Cifras de transposição

Exemplo: *rail fence*

- O texto em claro é escrito em uma diagonal em trilhos alternados de uma cerca que ao atingir o último trilho inverte a direção
  - Chave = número de trilhos (linhas)
  - Comprimento da cerca = número de letras do texto em claro
- O texto cifrado é obtido lendo-se as linhas individuais e concatenando-as
  - Escreve-se cada letra do texto em claro em linhas alternadas e depois lê-se em linhas

S				R				A
	E		U		A		Ç	
		G				N		

```
Mensagem: SEGURANÇA
chave:    3


Texto cifrado: SRAEUAÇGN
```

Criptografia moderna

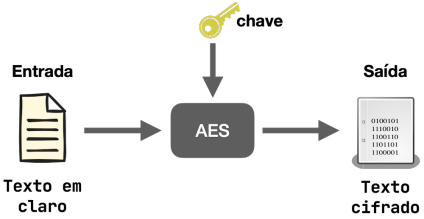
Criptografia moderna

Possível com o uso de computadores

- Além **confidencialidade**, a criptografia moderna tem como objetivo **garantir a autenticidade e a integridade** das mensagens
  - Baseada em problemas matemáticos difíceis de resolver
- **Algoritmos criptográficos são públicos e amplamente estudados**
  - Segurança é garantida pela dificuldade de resolver um problema matemático e pelo sigilo da chave
- **Criptosistema simétrico**
  - A mesma chave é usada para cifrar e decifrar a mensagem
- **Criptosistema assimétrico**
  - Duas chaves diferentes são usadas para cifrar e decifrar a mensagem

Aplicações de criptosistema simétrico

- **Armazenamento seguro de dados**
  - Arquivo individual, disco rígido ou partição, banco de dados
  - Exige que a chave secreta seja armazenada de forma segura
- **Comunicação segura entre duas partes**
  - Garante a confidencialidade, a autenticidade e a integridade das mensagens trocadas entre duas partes mesmo sobre um canal inseguro
  - A chave precisa ser compartilhada entre as partes previamente



Aplicações de criptosistema assimétrico

- **Comunicação segura entre duas partes**
  - E-mail seguro (PGP, S/MIME)
  - Comunicação segura em redes sociais
- **Assinatura eletrônica de documentos**
  - Contratos, documentos fiscais, etc.
- **Certificados digitais**
  - Parte da Infraestrutura de Chaves Públicas (ICP), usados para autenticar a identidade de uma entidade
- **Troca de chaves de sessão**
  - Criptografia simétrica é mais eficiente que a assimétrica
  - Criptografia assimétrica é usada para trocar chaves simétricas

Criptosistema assimétrico

Algoritmo RSA

- Proposto em 1977 por Ron Rivest, Adi Shamir e Leonard Adleman (RSA) é um dos mais usados atualmente
  - Baseado em aritmética modular, conjuntos finitos de números inteiros aos quais são aplicadas operações aritméticas básicas (adição, subtração, multiplicação e divisão)
- Baseado na dificuldade de fatoração do produto de dois números primos de grandes dimensões
  - É fácil multiplicar dois números primos $n = p \cdot q$
  - É difícil fatorar o resultado (calcular p e q a partir de n)
- O tamanho da chave é medido em bits e o tamanho mínimo recomendado atualmente é de 2048 bits

Par de chaves RSA de 2048 bits

- Chave privada


```
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQDlQHK3p5
BWQnlIZ3d6/0WVBt3bEb/SahBO7jxvff0yEW1d687uuv9IPeoXr3z0Jbgs
MLIWwBiFG1X0vJorhtDOyxKemHy4cO4dgSOJaZmYVq9fEa+e6CUDF6AKAs
OJKycDUGdJw59KmPrYuYAiUfZplmEs7JKVzFVeaaYIfJe2i7DxBJqZsRMf
GoRSMRlIvKpu2E3Q6tEG+e3tkG2S0Ee1q/NY57RiYT9Kz4lHyRNhq5Y6X2
WZDUeJXp3AyLWD2rhb4drMXoosnhmSUhlrVQIFBxJstpxOjDDO6TS56dBQ
WEc/SegYAszS4z1sGVZNdXcW6Vd8Oy6cMDmlpxSBo9MVAgMBAAECggEAEZ
d5dhhYnhXpNm+THpAaIc5uFdJt2tKiv7vOTWvokv/9WXiv32fgEQYXQYwl
1Z9TW6i88bv55bTsSqRyXCazPTseCad/4pNf4km3lWLSE3uLZ4su483CkV
Mtx7bglFUIYFu6IVHulkXY8eqCj6AMfZLSzCS3NuGAJ4eJ4xrUdYBrfENZ
/WmUJpkQWuY2B6OHU2x58SgCKZy649megp63RcQCuO19uzmJnjq1ipX0wU
4ES331DFdbjjKZhL9T3wRyH+5mMv3S6UnCHOL+Xdkxl9T1bVedHWCf7pMZ
bQ4ORZJkRsNAHoo+2fYccuQbULaIH02JOIKu5d97MGwRxKi6PQKBgQDz1J
r7UWVdespScGKG6ksGPZ4X7b+hIMYe8nGRnhqGWaQDcT867OJIwMDmS0Rs
0+KQnqkSQCo0PvmOsz1/MvPP7AOWgKha3nY8OPWK9Sog2/Ng+b8W+qvENk
H3JuLLYYncVsLy0VrRm9uYy1b9CTdxQOE21wUCIHUoDyU44IvU4wKBgQDw
sZjIT37KC7CcYG3UFaTnfaU+nFzamif5Gb1BpsNnO8s726I7A2ebE0ecWp
CYirEQ7yHcHPOS4WvX9M/Z6AWlroFb5UpFi4NE6EvrmpL1EBsCoHlA9ofV
jT/ASjWHksmj6ez6LGvx1oQvphNt8W2Tgy0bzpohQYb9bz8csTmxpwKBgH
no38At9y9g2E2VkT2BYRdysVunbDtkhoosj6nN0ddAdGegNIwCgjTT4t6B
3WIIGyxRl5jfafqLTf0SEvpJAGwBxd124DXmqlj75ZCfeXvKXQosdGVJ2Z
2Vvy+SSqyDTQlUe84p7G2GrqnbaNfExuqiFLh1kzR4A/uObv7c9isfAoGB
AKzGitHPRrA6W/ikr7lCJ3GUm72p+G1jIzaI53XtA29jWKwLKRU7iG+lE0
6FJC6bwCbrwV6D2cSh1AwdS+abyxH3hhw1J29sijo4leb1+m+3s+vaza94
SGFfF1gs7Of40o27KcUbhhD/b7vUdzYAjCvLieHgGHuu0fuyk5dBEK0ZAo
GAXApi9ae5bZSHepys9H3kMSkcla4f6NgsBoDxytow1HOQrQwny4SZphwM
7DFZR45kX9tPa6jqPqKEFmcMd88LA6SHepHjt4d0fMtLJdEfDqZYUD0Enm
uV2Ef5gmI47KWhzx0ExHlIxElwQfmvQnP2Qqm8klx76Dc0Uu5gLi2pGk8=
-----END PRIVATE KEY-----
```

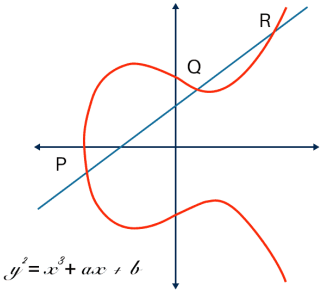

- Chave pública


```
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA5UByt6eQVkJyJW
d3ev9F1Qbd2xG/0moQTu48b339MhFtXevO7rr/SD3qF6989CW4LDCyFsAY
hRtV9LyaK4bQzssSnph8uHDuHYEjiWmZmFavXxGvnuglAxegCgLDiSsnA1
BnScOfSpj62LmAIlH2aSJhLOySlcxVXmmmCHyXtouw8QSambETHxqEUjES
JbyqbthNOOrRBvnt7ZBtktBHtavzWOe0YmE/Ss+JR8kTYauWOl9lmQ1HiV
6dwMi1g9q4W+HazF6KLJ4ZklISK1UCBQcSbLacTowwzuk0uenQUFhHP0no
GALMOuM9bBlWTXV3FulXfDsunDA5pacUgaPTFQIDAQAB
-----END PUBLIC KEY-----
```

Criptosistema assimétrico I

Curva Elíptica Criptográfica (ECC) vs RSA

- RSA baseia-se na dificuldade de fatorar o produto de dois números primos grandes
  - Dado dois números primos p e q, calcular $n = p \times q$ é fácil
  - Dado n é difícil encontrar p e q
- ECC baseia-se na dificuldade de resolver o problema do logaritmo discreto entre pontos em uma curva elíptica, mesmo que um dos pontos seja conhecido
- Chaves menores em ECC fornecem o mesmo nível de segurança que chaves maiores em RSA



Fonte: https://avinetworks.com/glossary/elliptic-curve-cryptography

Criptosistema assimétrico II

Curva Elíptica Criptográfica (ECC) vs RSA

Tabela: Comparação do tamanho das chaves para garantir o mesmo nível de segurança

Tamanho da chave (bits)		
Simétrica	**RSA**	**ECC**
80	1.024	160
112	2.048	224
128	3.072	256
192	7.680	384
256	15.360	521

- ECC é mais eficiente que RSA
- ECC é mais resistente a ataques quânticos
  - RSA é vulnerável a ataques baseados no algoritmo de Shor
- Até 2030, *draft* do NIST[a] recomenda chaves de 256 bits para ECC e 2.048 bits para RSA

[a] https://csrc.nist.gov/pubs/sp/800/78/5/ipd

Par de chaves ECC de 256 bits

Curva Elíptica Criptográfica prime256v1

- Chave privada


```
-----BEGIN EC PRIVATE KEY-----
MHcCAQEEIEABStAz2WkyjzXHd7SknEbvZ8ZAvEZeWLisrdWXnhxxoAoGCCqGSM49AwEHoUQDQgAEuHb4bzVS2hhqGeXNkm9EaHuTmU9KNBy5HuQR
8klQAdZzOSzRpy7q9IsezKOyckuPzzgeDR928FevebG9ot6ecA==
-----END EC PRIVATE KEY-----
```


- Chave pública


```
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEuHb4bzVS2hhqGeXNkm9EaHuTmU9KNBy5HuQR8klQAdZzOSzRpy7q9IsezKOyckuPzzgeDR928Fev
ebG9ot6ecA==
-----END PUBLIC KEY-----
```

Criptografia depende de entropia I

Entropia na computação

Medida da incerteza associada a um número aleatório. Quanto maior a entropia, mais imprevisível é o número aleatório

- Boas fontes de entropia estão associadas a processos físicos imprevisíveis
  - Ruído atmosférico ou térmico
  - Variação de atraso (*jitter*) de relógio
  - Interação de usuário (movimento do mouse, teclas pressionadas)
  - Interrupções de hardware, como movimento de disco rígido, tráfego de rede

Criptografia depende de entropia II

- A segurança de um sistema criptográfico **depende da qualidade dos números aleatórios** que são usados
  - Na geração de chaves criptográficas
  - *Salts* para senhas com *hash*
  - Número de uso único (*nonce*)

> **Nota**
>
> **Entropia é um recurso finito** e pode ser esgotado, gerando um bloqueio momentâneo na execução da aplicação por falta de entropia

Gerador de números aleatórios

Fonte: https://dilbert.com/

- ***Pseudo-Random Number Generator* (PRNG)**
  - Números pseudoaleatórios gerados a partir de algoritmos determinísticos, como o *Gerador Congruencial Linear* (GCL)
  - Não adequados quando se exige segurança criptográfica
- ***Cryptographically Secure Pseudo-Random Number Generator* (CSPRNG)**
  - Números aleatórios criptograficamente seguros gerados a partir de fontes de entropia de alta qualidade
  - Pode ser mais lento que o PRNG, pois depende de fontes de entropia
- ***True Random Number Generator* (TRNG)**
  - Números aleatórios verdadeiros gerados por um hardware especializado
  - Geralmente usado em aplicações de segurança crítica

Gerador de números pseudoaleatórios (PRNG)

- Algoritmo, derivado de uma função matemática, que gera sequências de números que parecem ser aleatórios, mas são determinísticos
- Não são adequados para gerar chaves criptográficas

```
// Classe Random gera números pseudoaleatórios de acordo com o
// modelo Gerador Congruencial Linear
// https://docs.oracle.com/javase/8/docs/api/java/util/Random.html

// Iniciando com a semente 123.
// Se não for fornecida, a semente é baseada no relógio do sistema
Random random = new Random(123);

// Será gerada sempre a mesma sequência: 82, 50, 76, 89 e 95
for (int i = 0; i < 5; i++) {
    System.out.println(random.nextInt(100));
}
```

Criptografia

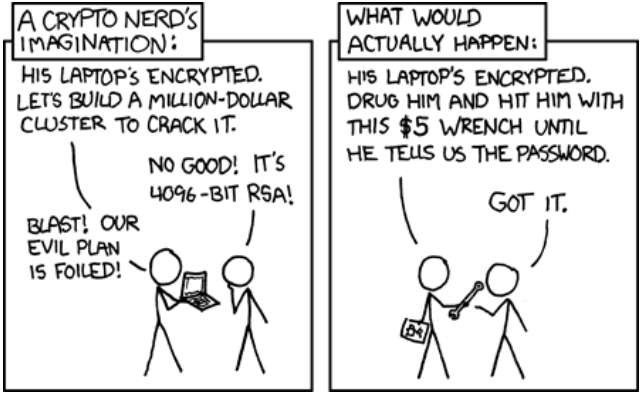
Implementações no Linux, Java e JavaScript

- No Linux o CSPRNG pode ser acessado via `/dev/random` com a chamada de sistema `getrandom`
- *Java Cryptography Architecture* (JCA)[2]
  - Conjunto de APIs para assinatura digital, criptografia simétrica e assimétrica, geração de chaves, *hash*, etc
  - Classe `SecureRandom`[3] usa o CSPRNG
- Em JavaScript o CSPRNG pode ser acessado via `window.crypto.getRandomValues(Uint8Array)` no navegador

[2] https://docs.oracle.com/en/java/javase/22/security/java-cryptography-architecture-jca-reference-guide.html

[3] https://docs.oracle.com/javase/8/docs/api/java/security/SecureRandom.html

Criptografia não resolve todos os problemas de segurança

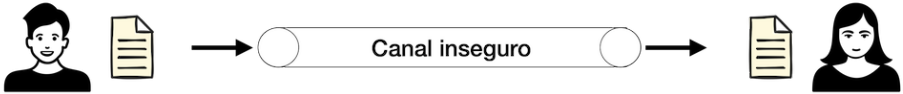


Fonte: https://xkcd.com/538

Comunicação segura entre partes

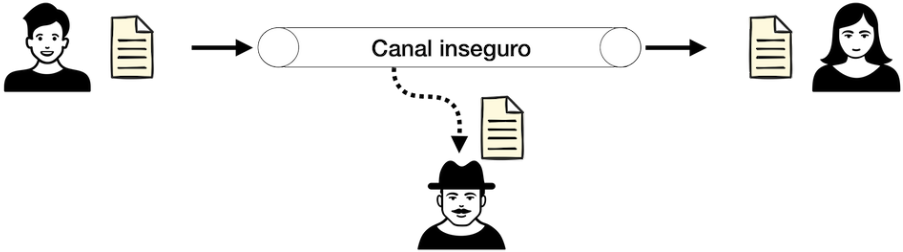
Comunicação segura entre duas partes

Criptografia simétrica



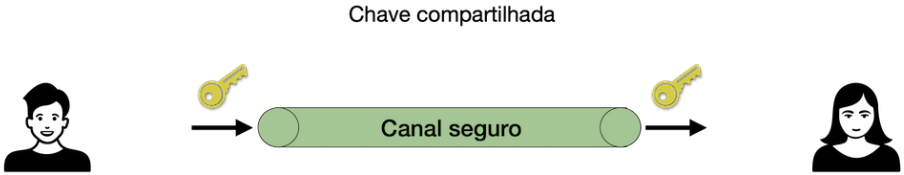
Comunicação segura entre duas partes

Criptografia simétrica



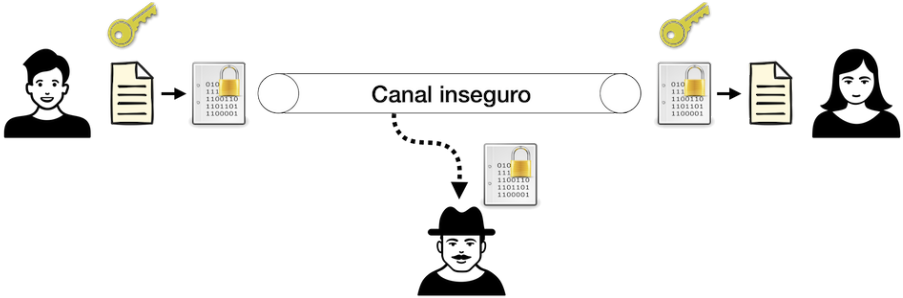
Comunicação segura entre duas partes

Criptografia simétrica



Comunicação segura entre duas partes

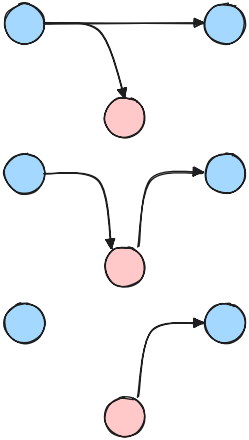
Criptografia simétrica



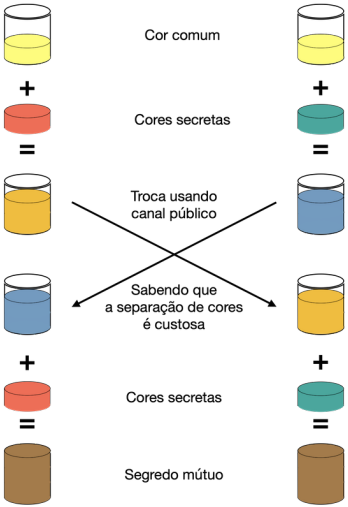
Como compartilhar a chave de forma segura?

Dificuldades com criptografia simétrica

1. **Encontro presencial**
   - Não é escalável
   - Distância, custo, tempo
2. **Envio por e-mail**
   - Não é seguro
   - Interceptação, MITM, personificação
3. **Uso de um terceiro confiável**
   - Como estabelecer essa confiança?
4. **Protocolo de acordo de chaves Diffie-Hellman**
   - Suscetível a ataques MITM



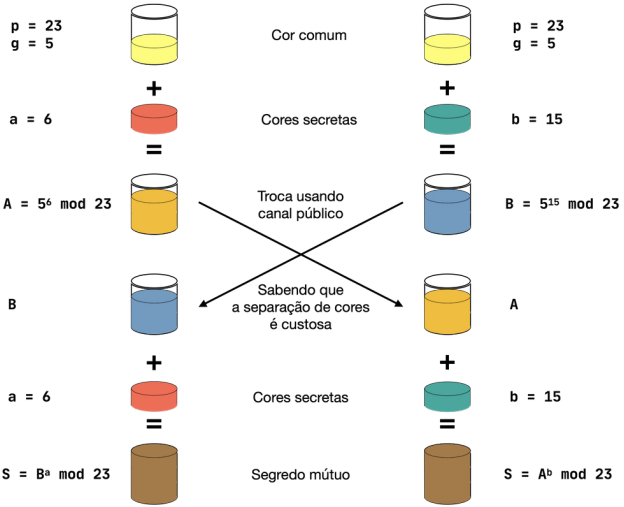
Protocolo de acordo de chaves Diffie-Hellman



Fonte: Adaptado de Wikipedia

Protocolo de acordo de chaves Diffie-Hellman

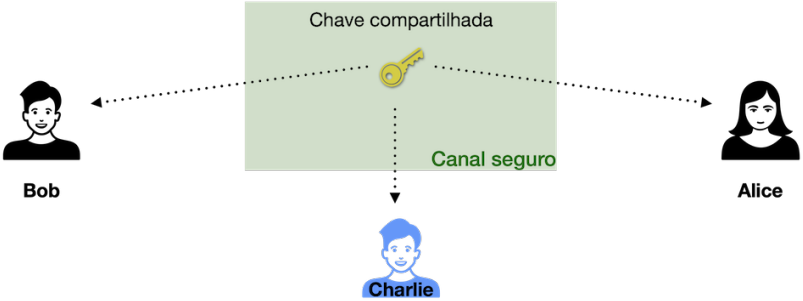
- Concordam com o primo $p = 23$ e como base $g = 5$
- O segredo de
  - Alice é $a = 6$
  - Bob é $b = 15$
- Alice calcula
  - $A = g^a \mod p$
  - $s = B^a \mod p$
- Bob calcula
  - $B = g^b \mod p$
  - $s = A^b \mod p$



Fonte: Adaptado de Wikipedia

E se for necessário conversar com mais de uma parte?

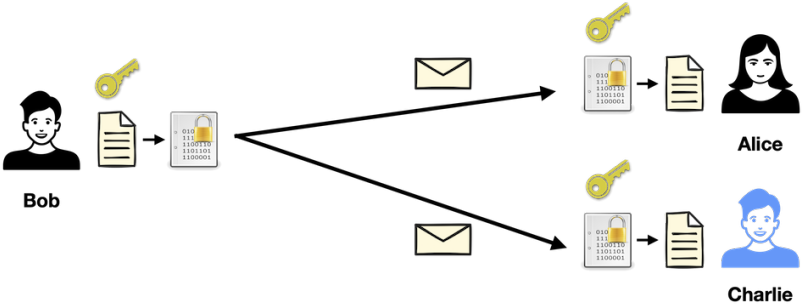
Dificuldades com criptografia simétrica



- É necessário compartilhar a mesma chave com cada uma das partes
- Se uma das partes for comprometida, todas as partes são comprometidas

E se for necessário conversar com mais de uma parte?

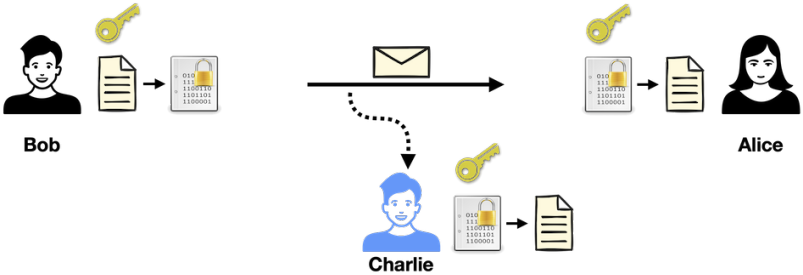
Dificuldades com criptografia simétrica



- É necessário compartilhar a mesma chave com cada uma das partes
- Se uma das partes for comprometida, todas as partes são comprometidas

E se for necessário conversar com mais de uma parte?

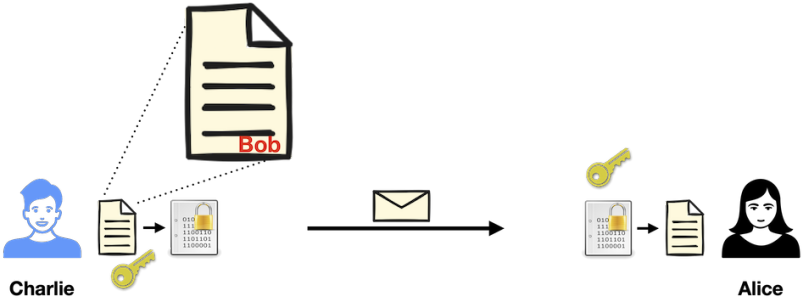
Dificuldades com criptografia simétrica



- Não é possível garantir a confidencialidade entre duas partes
- Todos que conhecem a chave podem decifrar a mensagem

E se for necessário conversar com mais de uma parte?

Dificuldades com criptografia simétrica



- Não é possível garantir a autenticidade da origem da mensagem
- Qualquer parte pode se passar por outra parte

Comunicação segura entre duas partes

Criptografia assimétrica

Garante a confidencialidade, a autenticidade e a integridade das mensagens

- **Cada parte possui um par de chaves: pública e privada**
  - A chave pública é compartilhada com todos
  - A chave privada é mantida em segredo
- **Para garantir a confidencialidade**
  - A chave pública é usada para cifrar a mensagem
  - A chave privada é usada para decifrar a mensagem
- **Para garantir a autenticidade**
  - A chave privada é usada para assinar a mensagem
  - A chave pública é usada para verificar a assinatura

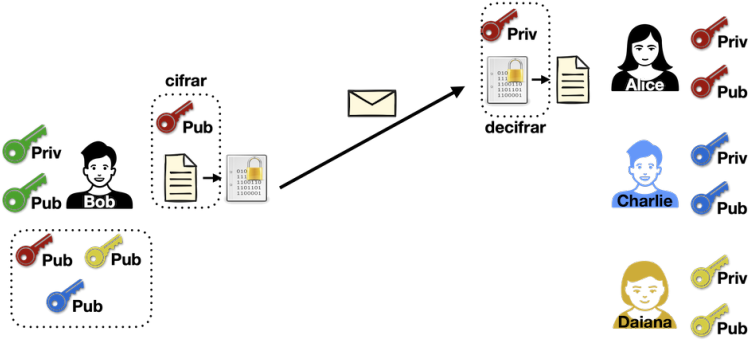
Comunicação segura entre duas partes

Criptografia assimétrica

- Bob mantém um chaveiro com as chaves públicas de seus contatos
- Todos os demais contatos podem manter seu próprio chaveiro

Comunicação segura entre duas partes

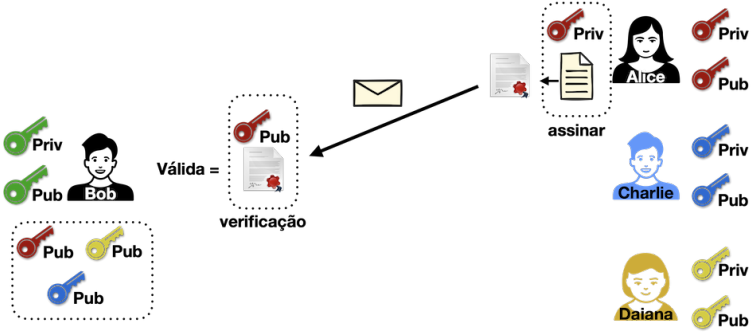
Criptografia assimétrica



- Bob cifra a mensagem com a chave pública de Alice
- Somente Alice pode decifrar a mensagem com sua chave privada

Comunicação segura entre duas partes

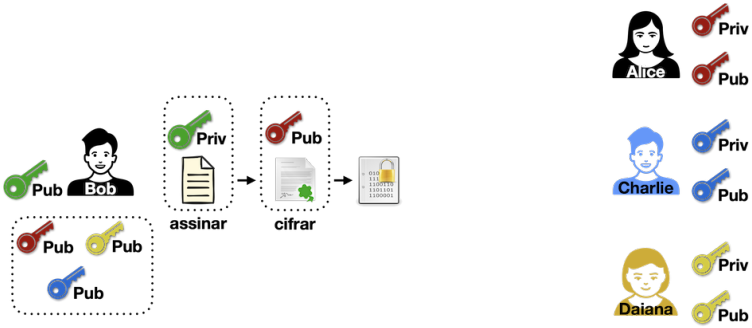
Criptografia assimétrica



- Alice assina a mensagem com sua chave privada
- Bob verifica a assinatura com a chave pública de Alice

Comunicação segura entre duas partes

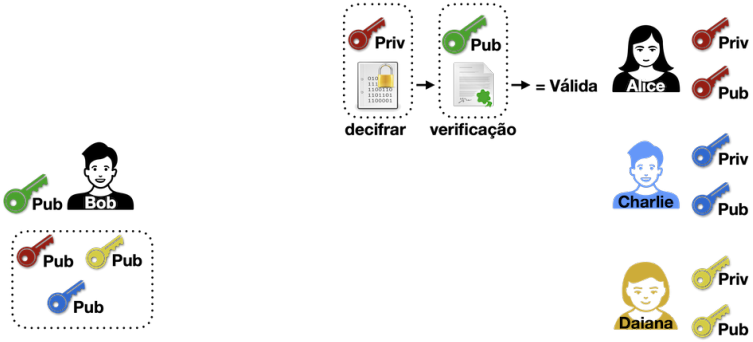
Criptografia assimétrica



- Bob assina a mensagem com sua chave privada e cifra com a chave pública de Alice

Comunicação segura entre duas partes

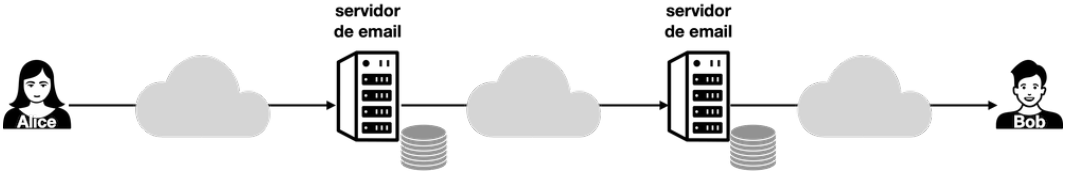
Criptografia assimétrica



- Alice decifra a mensagem com sua chave privada e verifica a assinatura com a chave pública de Bob

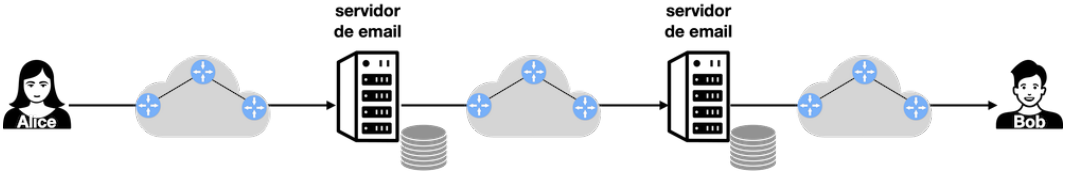
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



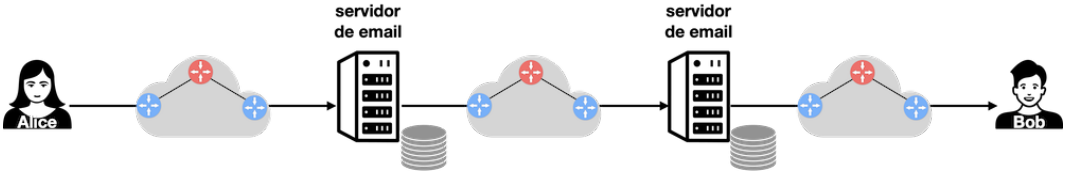
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



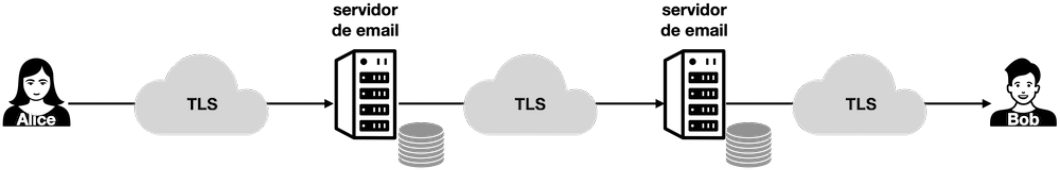
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



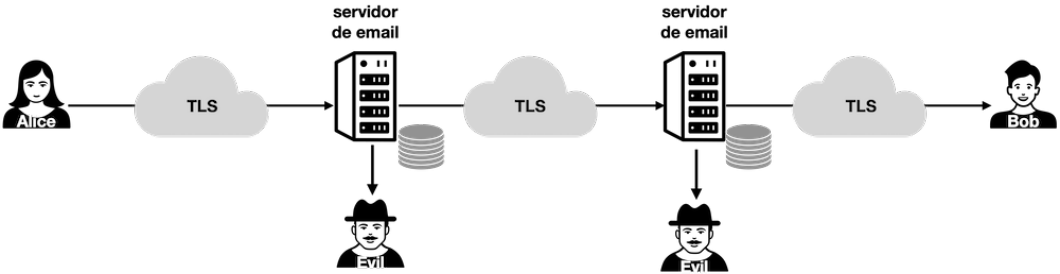
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



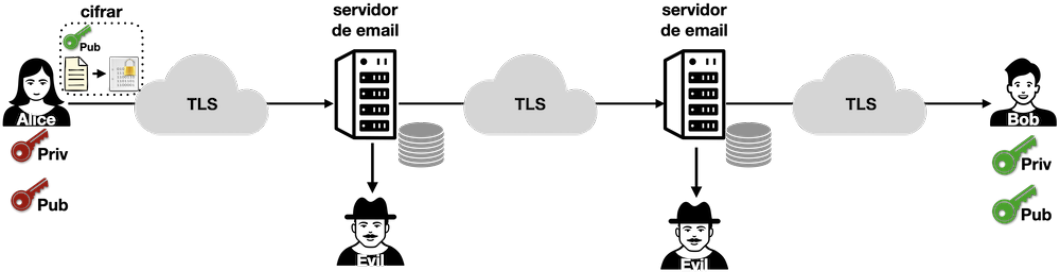
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



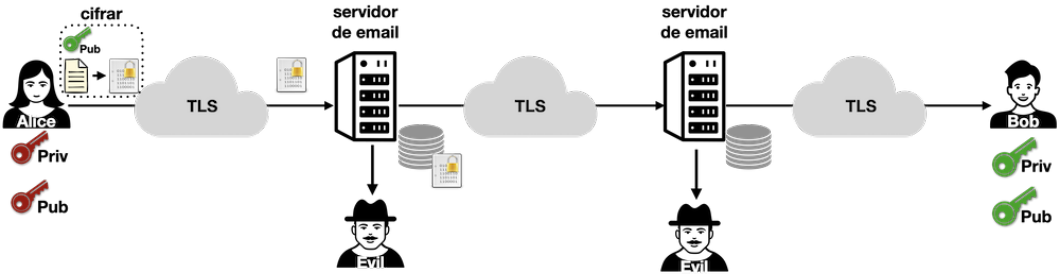
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



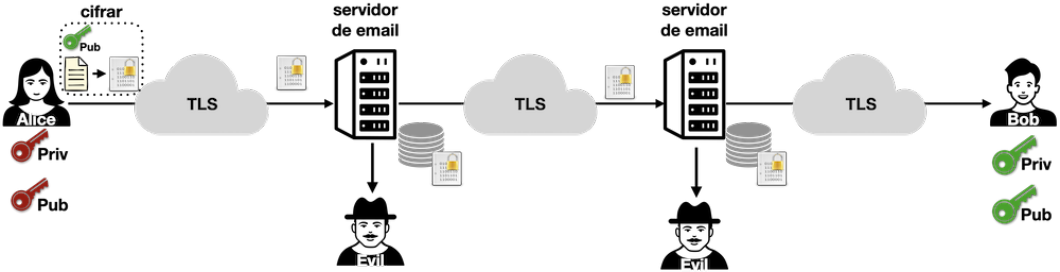
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



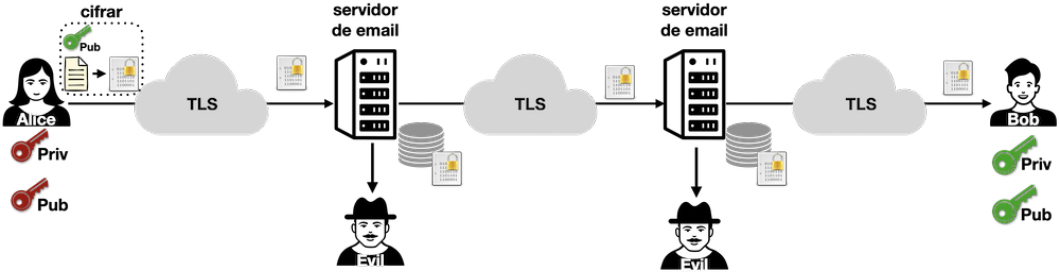
Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



Criptografia fim-a-fim (E2EE, *end-to-end encryption*)

Garante que o conteúdo só possa ser acessado pelo remetente e o destinatário



Função de dispersão criptográfica

Integridade

Garantia de que a informação não foi alterada de forma não autorizada

- Como detectar que um arquivo foi corrompido?
- Como detectar que dados foram alterados de forma não autorizada?

Integridade

Garantia de que a informação não foi alterada de forma não autorizada

- Como detectar que um arquivo foi corrompido?
- Como detectar que dados foram alterados de forma não autorizada?

⚠ Atenção

O *checksum* do pacote IP detecta erros de transmissão no cabeçalho, mas não garante integridade do conteúdo do pacote nem é seguro contra manipulação

Funções de dispersão criptográfica

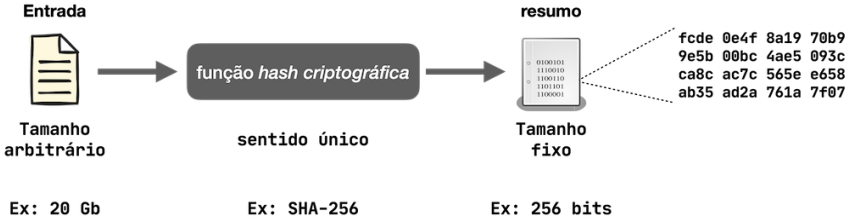
Aplicações

- **Autenticação de usuários**
  - Senhas não são armazenadas em texto claro, mas como *hash*
- **Autenticação de mensagens**
  - Garantir que a mensagem não foi alterada e é autêntica
- **Integridade ou identificador de arquivos**
  - Ex: imagens ISO; versões no Git
- **Assinaturas digitais**
  - Garantir que um documento é autêntico e não foi alterado
- **Prova de trabalho (mineração de Bitcoins)**
  - Achar um *hash* que começa com um número mínimo de zeros

Função de dispersão criptográfica

Cryptographic hash function

- Mapeia entrada de tamanho arbitrário para uma saída de tamanho fixo
  - A saída é chamada de *hash* ou *resumo criptográfico*
- Função unidirecional, não é possível recuperar a entrada a partir do *hash*



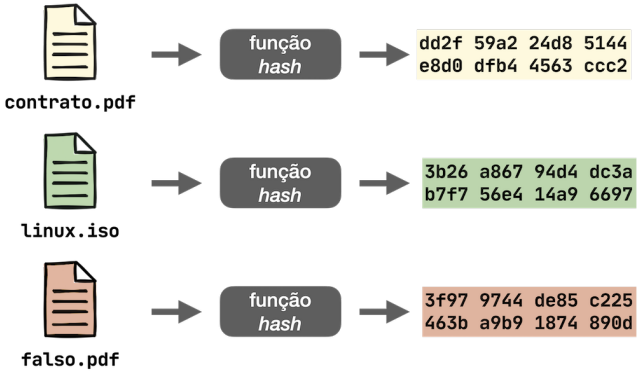
Funções de dispersão criptográfica

Propriedades

- **Determinismo**
  - A mesma entrada sempre produz o mesmo *hash*
- **Resistência a pré-imagem** (sentido único)
  - Dado um *hash* h, deve ser computacionalmente inviável encontrar uma entrada m tal que $h = \text{hash}(m)$
- **Resistência a segunda pré-imagem** (resistência a colisão fraca)
  - Dado uma entrada m_1, deve ser computacionalmente inviável encontrar uma entrada m_2 diferente de m_1 tal que $\text{hash}(m_1) = \text{hash}(m_2)$
- **Resistência a colisões**
  - Deve ser computacionalmente inviável encontrar duas entradas diferentes, m_1 e m_2, tal que $\text{hash}(m_1) = \text{hash}(m_2)$
- **Difusão**
  - Uma pequena alteração na entrada deve produzir um *hash* completamente diferente

Funções de dispersão criptográfica

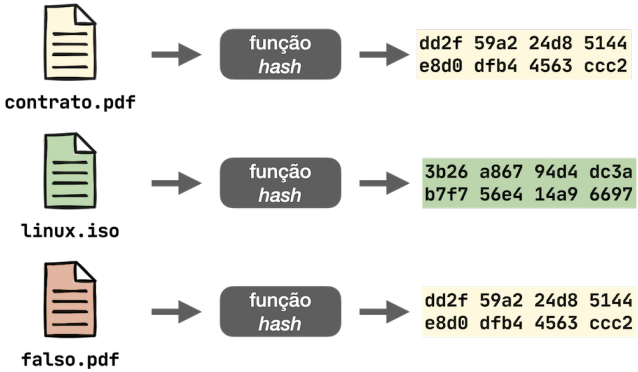
Colisões permitiriam um adversário substituir ou modificar uma informação sem ser detectada



- Entradas diferentes devem produzir *hashes* diferentes
- Colisões são possíveis, mas devem ser computacionalmente inviáveis de serem encontradas

Funções de dispersão criptográfica

Colisões permitiriam um adversário substituir ou modificar uma informação sem ser detectada

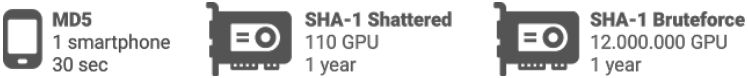


- Entradas diferentes devem produzir *hashes* diferentes
- Colisões são possíveis, mas devem ser computacionalmente inviáveis de serem encontradas

Algoritmos de funções de dispersão criptográfica

Algoritmo	Resumo (bits)	Comentários
MD5	128	Inseguro, usado na NF-e no Brasil
SHA-1	160	Inseguro, usado no git e torrents
SHA-2	224 a 512	Bitcoin usa SHA-256
SHA-3	256 e 512	Padrão NIST

- Em 2017 a Google apresentou um ataque de colisão prático para o SHA-1
  - É possível criar dois PDFs diferentes com o mesmo *hash*

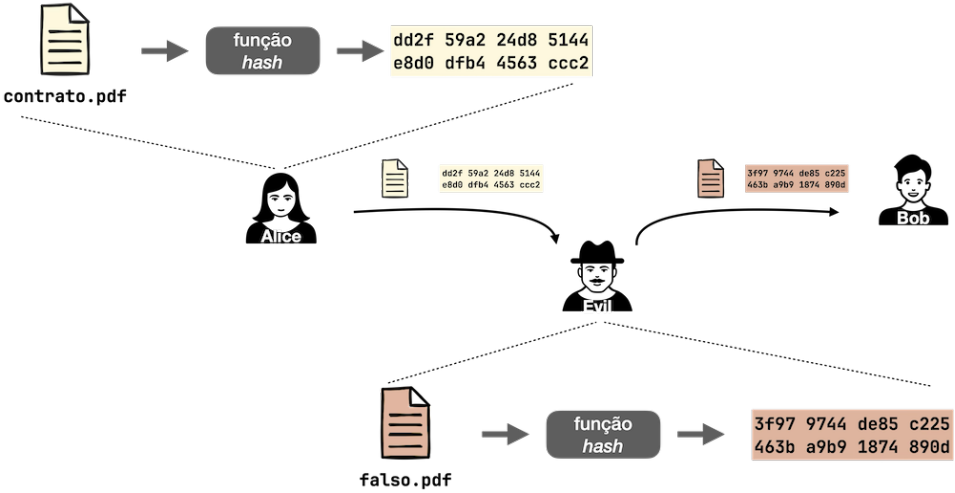


Fonte: https://shattered.io

Hash-based Message Authentication Code (HMAC) I

- Funções de dispersão criptográfica (*hash*) garantem integridade, mas não autenticidade
- Um adversário pode gerar o *hash* de uma mensagem modificada e enviar como se fosse legítima

Hash-based Message Authentication Code (HMAC) II



Hash-based Message Authentication Code (HMAC) III

HMAC

- Forma de autenticação de mensagens que combina uma chave secreta com uma função de resumo criptográfico
- Apenas quem conhece a chave secreta pode gerar ou verificar o HMAC corretamente
- Permite verificar a autenticidade e a integridade de uma mensagem, mas não garante a confidencialidade

HMAC

Funcionamento

1. Emissor e receptor compartilham uma chave secreta K
2. Emissor calcula o HMAC da mensagem M usando K e envia:

$$\text{HMAC}(K, M) = \text{hash}\left((K \oplus \text{opad}) \parallel \text{hash}((K \oplus \text{ipad}) \parallel M)\right) \tag{3}$$

   , onde *hash* é a função de resumo criptográfico, K é a chave secreta, M é a mensagem, opad e ipad são preenchimentos[4] externo e interno, $\oplus$ é XOR e $\parallel$ é concatenação.

3. Receptor, ao receber M e o HMAC, calcula $\text{HMAC}(K, M)$ e compara com o valor recebido
4. Se os valores coincidirem, a mensagem é autêntica e íntegra

[4] Necessários para adequar o tamanho da chave com o tamanho do bloco da função de hash.

HMAC

Exemplo com OpenSSL

- Emissor gera o HMAC de um arquivo usando uma chave secreta e o envia junto com o arquivo

```
# Gerar HMAC-SHA256 de um arquivo usando a chave 'segredo'
openssl dgst -sha256 -hmac segredo contrato.pdf

# Saída:
HMAC-SHA2-256(contrato.pdf)= bc252b0cd3da5a85...
```

- O receptor pode repetir o comando com a mesma chave para verificar a autenticidade e integridade do arquivo

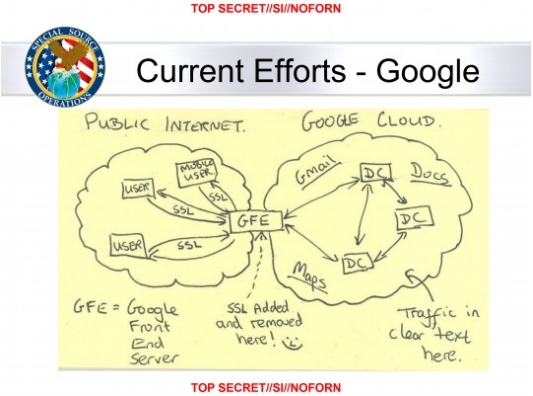
Exemplo em Java para gerar um HMAC

```java
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.util.Base64;

public class ExemploHmac {

  public static void main(String[] args) throws Exception {
    String chave = "sua chave secreta";
    String dados = "Aula de Segurança no ADS";
    SecretKeySpec keySpec = new SecretKeySpec(chave.getBytes(StandardCharsets.UTF_8), "
     HmacSHA256");
    Mac mac = Mac.getInstance("HmacSHA256");
    mac.init(keySpec);
    byte[] hmacBytes = mac.doFinal(dados.getBytes(StandardCharsets.UTF_8));
    System.out.println("HMAC: " + Base64.getEncoder().encodeToString(hmacBytes));
  }
}

// Saída: HMAC: uxgrTn9UW+RyYl1VuV/7lqaKbC04xRPwYRv+vv7cvIg=
```

Curiosidades

Programa de vigilância MUSCULAR, parceria entre a NSA e a GCHQ (Reino Unido)

- Em 2013, Edward Snowden revelou que dados eram interceptados entre os centros de dados da Google e Yahoo



Fonte: *U.S. National Security Agency*

Curiosidades

Codificação em BASE64

- Transformar um texto em BASE64[5] não é uma forma de criptografia
- BASE64 é uma forma de codificar dados binários em ASCII
- A codificação BASE64 é usada para transmitir dados binários em protocolos de comunicação como HTTP, SMTP, POP3, IMAP, FTP, etc.

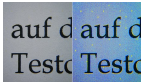
```
echo -n "ADS IFSC" | base64
# Saída: QURTIElGUOM=
```

[5] https://www.base64encode.org

Curiosidades

Esteganografia

- Prática de esconder mensagens dentro de outras mensagens
  - Usada para esconder uma mensagem em pixels menos significativos em uma imagem, em um vídeo, em um áudio, etc.
- Em 2004 foi revelado que todas impressoras laser coloridas produzem um código de identificação invisível a olho nu
  - Matriz de pontos amarelos que identifica a impressora e a data e hora da impressão



Na imagem da árvore, a esteganografia foi usada para esconder a imagem do gato

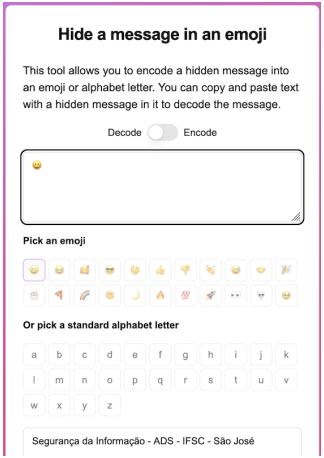
Fonte: Wikipedia

Curiosidade

Escondendo dados em caracteres unicode

- Acesse https://emoji-encoder.vercel.app para codificar e decodificar mensagens
- Paul Butler[a] apresenta uma explicação sobre como esconder dados em caracteres unicode

[a] https://paulbutler.org/2025/smuggling-arbitrary-data-through-an-emoji



Referências

- Os ícones representando as pessoas foram obtidos de https://uxwing.com